

Prompt: what are the implementations(e.g_ Create the MySQL database schema for these resume features)

To implement **US-04 Resume Upload**, your development team will need to coordinate database changes, file storage configuration, API routes, and frontend components [cite: 1].

Here is the technical implementation breakdown for this specific issue:

1. Database Implementation (MySQL Schema)

To track uploaded resumes, you need to extend your database. Since resumes are linked to a specific user, you can create a resumes table with a foreign key pointing to your existing users or profiles table.

```
CREATE TABLE resumes (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  user_id INT NOT NULL,  
  file_name VARCHAR(255) NOT NULL,      -- e.g., "John_Doe_Resume.pdf"  
  file_path VARCHAR(512) NOT NULL,      -- The storage URL or folder path  
  file_size_bytes INT NOT NULL,          -- To enforce the 5MB limit  
  uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE  
  CURRENT_TIMESTAMP,  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE  
);
```

2. Storage Implementation

- **Storage Solution:** Resumes should not be stored directly as binary blobs in the MySQL database (as this slows down performance). Instead, store the physical .pdf file in a dedicated cloud bucket (like **Amazon S3**, **Google Cloud Storage**) or a secured upload folder on your server.
- **Path Mapping:** The backend will upload the file to storage, generate a unique file path string, and save that string into the file_path column of the MySQL table shown above.

3. Backend API Implementation (REST Endpoints)

The backend developer will need to build three primary API routes:

- **POST /api/resumes/upload**
 - **Payload:** multipart/form-data containing the PDF file.
 - **Logic:** Validates that the file type is strictly application/pdf and under 5MB. Uploads it to storage, writes the record to MySQL, and returns a success response.
- **GET /api/resumes**

- **Logic:** Fetches the resume record linked to the logged-in user's ID so the frontend can display the file name.
- **DELETE /api/resumes/:id**
 - **Logic:** Deletes the physical file from the storage bucket and removes the row from the MySQL database.

4. Frontend UI Implementation

The user interface requires three main states:

- **Empty State:** Displays a "Drag and Drop your PDF here" area or an "Upload Resume" button.
- **Uploading State:** Disables the button and shows a loading spinner or progress bar.
- **Completed State:** Displays the uploaded file name with a paperclip icon, a download link, and a "Delete/Replace" button.

Would you like me to provide:

- The exact **Node.js/Python code** to handle the file validation and upload?
- The **React or HTML/CSS code** to build the frontend drop-zone area?
- A breakdown of how to transition this directly into **Story 5 (Multiple Resumes)** later?